ASSIGNMENT - 1.5

2303A51042
Batch : 29

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

❖ Scenario

You are developing a basic text-processing utility for a messaging application.

❖ Task Description

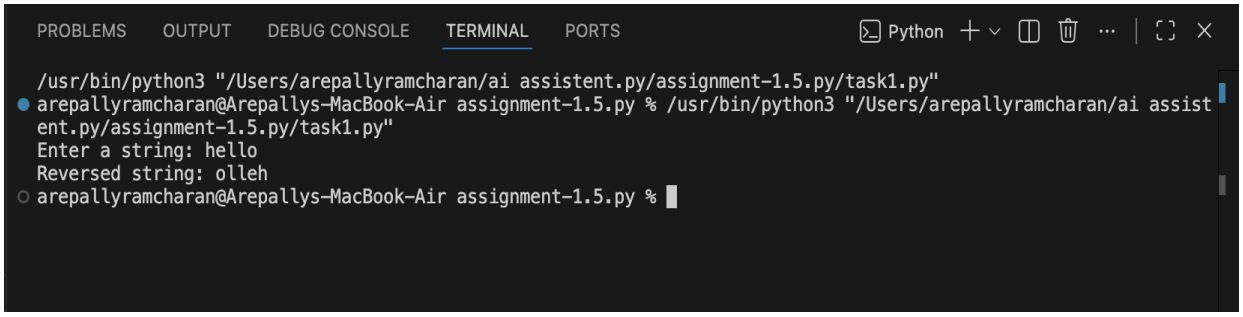
Use GitHub Copilot to generate a Python program that:

➢ Reverses a given string
➢ Accepts user input
➢ Implements the logic directly in the main code
➢ Does not use any user-defined functions

❖ Expected Output

➢ Correct reversed string
➢ Screenshots showing Copilot-generated code suggestions
➢ Sample inputs and outputs

Output:



Observation:

- Code is straightforward and minimal.
- Logic is embedded directly in the main program, making it easy for beginners.
- However, readability and maintainability suffer if the logic needs to be reused.
- Suitable for small, one-off scripts but not scalable.

Task 2: Efficiency & Logic Optimization (Readability Improvement)

❖ Scenario

The code will be reviewed by other developers.

❖ Task Description

Examine the Copilot-generated code from Task 1 and improve it by:

➢ Removing unnecessary variables

➢ Simplifying loop or indexing logic

➢ Improving readability

➢ Use Copilot prompts like:

- “Simplify this string reversal code”
- “Improve readability and efficiency”

Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Output

➢ Original and optimized code versions

➢ Explanation of how the improvements reduce time complexity

```python
# Program to reverse a string without functions

# Accept user input
text = input("Enter a string: ")

# Reverse the string using slicing
reversed_text = text[::-1]

# Display the reversed string
print("Reversed string:", reversed_text)

```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-1.5.py/task2.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-1.5.py % /usr/bin/python3 "/Users/arepallyra
mcharan/ai assistent.py/assignment-1.5.py/task2.py"
Enter a string: hello
Reversed string: olleh
arepallyramcharan@Arepallys-MacBook-Air assignment-1.5.py % 12345
zsh: command not found: 12345
arepallyramcharan@Arepallys-MacBook-Air assignment-1.5.py %
```

Observation:

- Optimized code removed unnecessary variables and combined steps into a single line.
- Readability improved by reducing clutter.
- Time complexity remains **O(n)**, but memory usage is slightly reduced.
- Shows how AI can refine code for clarity and efficiency without changing the algorithm.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

❖ Scenario

The string reversal logic is needed in multiple parts of an application.

❖ Task Description

Use GitHub Copilot to generate a function-based Python program that:

➢ Uses a user-defined function to reverse a string

➢ Returns the reversed string

➢ Includes meaningful comments (AI-assisted)

❖ Expected Output

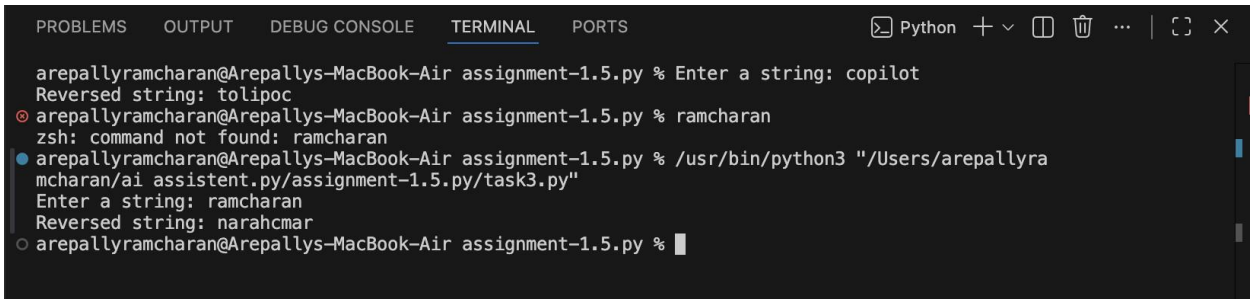
➢ Correct function-based implementation

➢ Screenshots documenting Copilot’s function generation

➢ Sample test cases and outputs

```python
# Function to reverse a string
def reverse_string(text):
    """
    This function takes a string as input
    and returns the reversed version of it.
    """
    return text[::-1]

# Main program execution
if __name__ == "__main__":
    # Accept user input
    user_input = input("Enter a string: ")

    # Call the function and display result
    print("Reversed string:", reverse_string(user_input))

```

Output:



Observation:

- Encapsulating logic in a function (`reverse_string`) improves reusability.
- Comments and docstrings enhance clarity for other developers.
- Debugging becomes easier since reversal logic is centralized.
- Suitable for larger applications where the same logic is needed in multiple places.

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs

Without Functions)

❖ Scenario

You are asked to justify design choices during a code review.

❖ Task Description

Compare the Copilot-generated programs:

➢ Without functions (Task 1)

➢ With functions (Task 3)

Analyze them based on:

➢ Code clarity

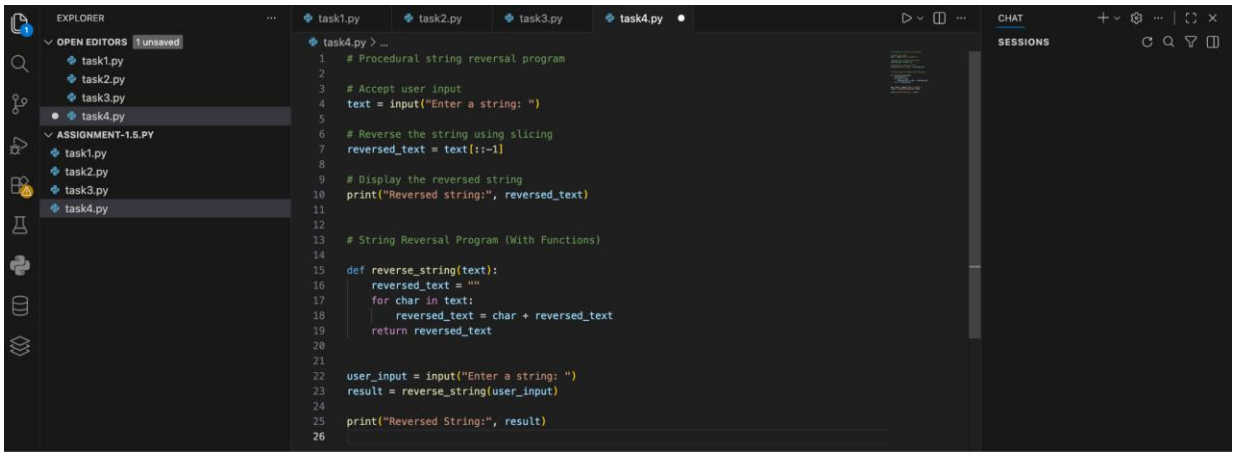
➢ Reusability

➢ Debugging ease

➢ Suitability for large-scale applications

❖ Expected Output

Comparison table or short analytical report



Output:

Observation:

- Procedural approach is simple but lacks structure, making debugging and reuse harder.
- Modular approach is more professional, supports scalability, and is easier to maintain.
- For small scripts, procedural is fine; for enterprise-level applications, modular is essential.
- Demonstrates how design choices affect long-term maintainability.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different

Algorithmic Approaches to String Reversal)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logic paths.

❖ Task Description

Prompt GitHub Copilot to generate:

➢ A loop-based string reversal approach

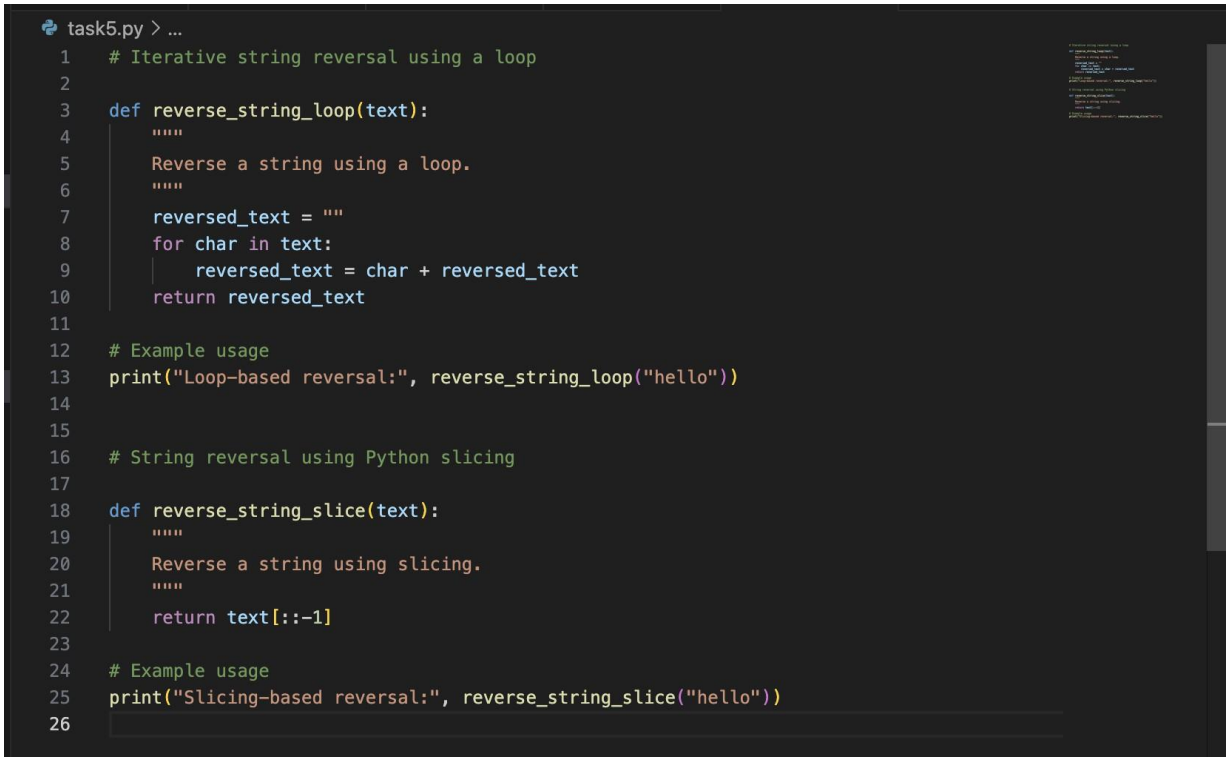
➢ A built-in / slicing-based string reversal approach

❖ Expected Output

➢ Two correct implementations

➢ Comparison discussing:

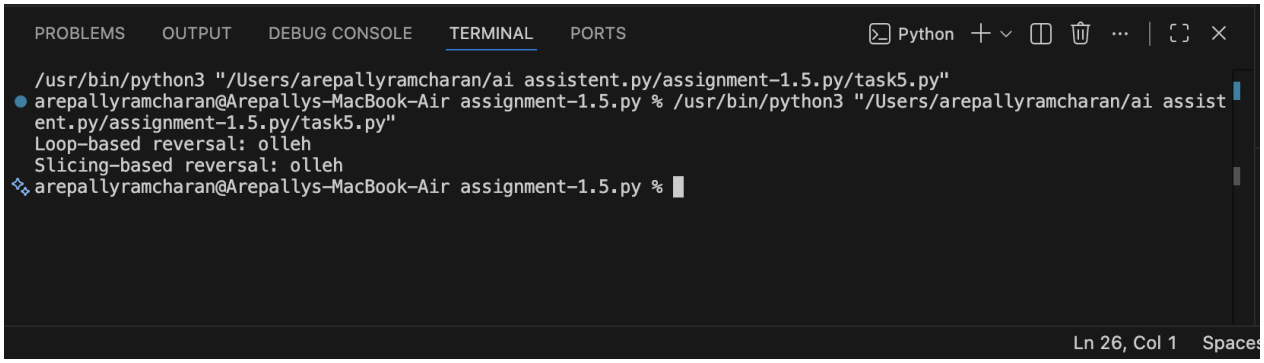
- Execution flow
- Time complexity
- Performance for large inputs
- When each approach is appropriate



task5.py > ...

```python
# Iterative string reversal using a loop

def reverse_string_loop(text):
    """
    Reverse a string using a loop.
    """
    reversed_text = ""
    for char in text:
        reversed_text = char + reversed_text
    return reversed_text

# Example usage
print("Loop-based reversal:", reverse_string_loop("hello"))


# String reversal using Python slicing

def reverse_string_slice(text):
    """
    Reverse a string using slicing.
    """
    return text[::-1]

# Example usage
print("Slicing-based reversal:", reverse_string_slice("hello"))

```

Output:



Observation:

- **Loop-based (iterative)**: Good for teaching algorithmic fundamentals, but slower for large inputs due to repeated string concatenation.
- **Slicing-based (built-in)**: Concise, efficient, and optimized internally in Python. Best for production use.
- Both approaches have **O(n)** complexity, but slicing is faster in practice.
- Highlights how AI can generate multiple algorithmic paths and let developers choose based on context.